

Banking Transaction Management System (BTMS)

BRD – Banking Transaction Management System (BTMS)

Database-Only Testing Scope

1. Document Purpose

This document defines the database requirements for the Banking Transaction Management System (BTMS).

It is intended for QA testers to validate:

- Data integrity
- Business rules
- Relational consistency
- Triggers and constraints

Testing is limited to INSERT and UPDATE operations only.

2. Project Scope

In-Scope:

- Customer accounts
- Financial transactions
- Transaction categories
- Branches and employees
- Fees and transaction limits
- Account balance management

Out-of-Scope:

- Front-end UI
- APIs
- Reporting dashboards

Testing Scope:

- INSERT and UPDATE queries only
- SQL validation of business rules
- Data integrity checks

3. Functional Requirements

FR-1 – Customer Management

- Add new customers
- Update customer information
- Mandatory fields: full_name, date_of_birth, email (unique), phone

Constraints:

- Email must be unique
- Phone must contain numeric values only

FR-2 – Account Management

- Customers can have multiple accounts
- Account types: Savings, Current, Wallet

Mandatory fields:

- account_number (unique)
- account_type
- balance ≥ 0

Business Rules:

- Savings and Current accounts cannot have negative balance
- Wallet accounts support instant transactions
- Account balance must update automatically after each transaction

FR-3 – Transaction Management

- Track all deposits, withdrawals, transfers, and bill payments

Mandatory fields:

- transaction_id (PK)
- sender_account
- receiver_account (nullable for withdrawals/deposits)
- transaction_type
- amount
- transaction_date

Transaction Types ENUM:

- Deposit
- Withdrawal
- Transfer
- Bill Payment

Constraints:

- amount > 0
- Sender account must exist
- Cannot transfer or withdraw more than available balance

Business Rules:

- Transactions must update balances automatically
- Transfers between accounts must occur atomically
- Failed transactions must not affect balances
- Daily transfer limits enforced via triggers

FR-4 – Transaction Categories

- Transactions can be assigned to categories

Mandatory fields:

- category_id
- category_name

Category ENUM Examples:

- Utilities
- Shopping
- Salary
- Entertainment
- Other

Business Rules:

- Each transaction may belong to one category
- Category updates should reflect in reporting consistency

FR-5 – Fees and Charges

- Auto-calculate fees for transfers and failed transactions

Mandatory fields:

- fee_id
- transaction_id
- fee_type
- amount

Fee Types ENUM:

- Transfer Fee
- Failed Transaction Fee
- Over Limit Fee

Business Rules:

- Transfer fee applies to external transfers
- Fee cannot exceed 10% of transaction amount
- Failed transaction fee applied only once per failed attempt

FR-6 – Employee and Branch Management

- Employees linked to branches

Branch Mandatory Fields:

- branch_id
- name
- location

Employee Mandatory Fields:

- employee_id
- full_name
- branch_id

Constraints:

- branch_id FK

Business Rules:

- Employee cannot be deleted if responsible for transaction approvals

4. Non-Functional Requirements

- All PKs auto-incremented
- FK relationships enforced
- CHECK constraints for numeric fields and ENUM values
- Triggers to update balances and apply transaction fees
- Performance: INSERT/UPDATE <= 2 seconds
- DB: MySQL / SQL Server
- Naming convention: snake_case

5. Assumptions

- One banking system instance
- Dates in YYYY-MM-DD format
- Transactions processed in real time
- Only INSERT/UPDATE operations performed by testers

6. Database Schema (SQL Creation)

6.1 Customers

```
CREATE TABLE Customers (  
    customer_id INT PRIMARY KEY AUTO_INCREMENT,  
    full_name VARCHAR(100) NOT NULL,  
    date_of_birth DATE NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    phone VARCHAR(20) NOT NULL  
);
```

6.2 Accounts

```
CREATE TABLE Accounts (  
    account_number INT PRIMARY KEY AUTO_INCREMENT,  
    customer_id INT NOT NULL,  
    account_type ENUM('Savings','Current','Wallet') NOT NULL,  
    balance DECIMAL(15,2) CHECK(balance >= 0),  
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)  
);
```

6.3 Transaction_Categories

```
CREATE TABLE Transaction_Categories (  
    category_id INT PRIMARY KEY AUTO_INCREMENT,  
    category_name VARCHAR(50) UNIQUE NOT NULL  
);
```

6.4 Transactions

```
CREATE TABLE Transactions (  
    transaction_id INT PRIMARY KEY AUTO_INCREMENT,  
    sender_account INT NOT NULL,  
    receiver_account INT,  
    category_id INT,  
    transaction_type ENUM('Deposit','Withdrawal','Transfer','Bill Payment') NOT NULL,  
    amount DECIMAL(15,2) CHECK(amount > 0),  
    transaction_status ENUM('Success','Failed','Pending') DEFAULT 'Pending',  
    transaction_date DATE NOT NULL,  
    FOREIGN KEY (sender_account) REFERENCES Accounts(account_number),  
    FOREIGN KEY (receiver_account) REFERENCES Accounts(account_number),  
    FOREIGN KEY (category_id) REFERENCES Transaction_Categories(category_id)  
);
```

6.5 Fees

```
CREATE TABLE Fees (  
    fee_id INT PRIMARY KEY AUTO_INCREMENT,  
    transaction_id INT NOT NULL,  
    fee_type ENUM('Transfer Fee','Failed Transaction Fee','Over Limit Fee') NOT NULL,  
    amount DECIMAL(10,2) CHECK(amount > 0),  
    FOREIGN KEY (transaction_id) REFERENCES Transactions(transaction_id)  
);
```

6.6 Branches

```
CREATE TABLE Branches (  
    branch_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    location VARCHAR(100)  
);
```

6.7 Employees

```
CREATE TABLE Employees (  
    employee_id INT PRIMARY KEY AUTO_INCREMENT,  
    full_name VARCHAR(100) NOT NULL,  
    branch_id INT NOT NULL,  
    FOREIGN KEY (branch_id) REFERENCES Branches(branch_id)  
);
```

7. Realistic Business Rules for DB Testing

- Transactions cannot exceed available balance
- Transfers must deduct and add balances correctly
- Failed transactions should not affect balances
- Daily transaction limits enforced
- Fees applied automatically via triggers
- Email uniqueness enforced
- Account balances updated via INSERT/UPDATE only
- Transaction status must be valid

8. Testing Guidelines

- INSERT sample data for Customers, Accounts, Transactions, Categories, Fees, Branches, Employees
- UPDATE queries to test business rules and constraints

Validate Using SELECT Queries:

- Account balances
- Transaction totals
- Fee calculations

- Failed transactions
- Daily transfer limits

Focus on Edge Cases:

- Negative amounts
- Duplicate emails
- Overdraft attempts
- Invalid transaction types
- Transfers exceeding daily limit
- Transactions with missing receiver accounts

9. Complexity & Sizing

- Tables: 7
- Constraints & business rules: 20+
- Triggers for balance updates and fees: 2
- Estimated test cases: 50–60 (positive + negative)
- Weighting: multi-table relationships → weight 2, simple field constraint → weight 1